Навчання з підкріпленням

Навчання з підкріпленням

Навчання з підкріпленням — це навчання того, що робити — як співставляти ситуації з діями — щоб максимізувати числовий сигнал винагороди.

Агенту не кажуть, які дії виконувати, натомість він повинен визначити, які дії приносять найбільшу винагороду, спробувавши їх.

У найбільш цікавих і складних випадках дії можуть вплинути не тільки на негайну винагороду, але й на наступну ситуацію, а через це й на всі подальші винагороди.

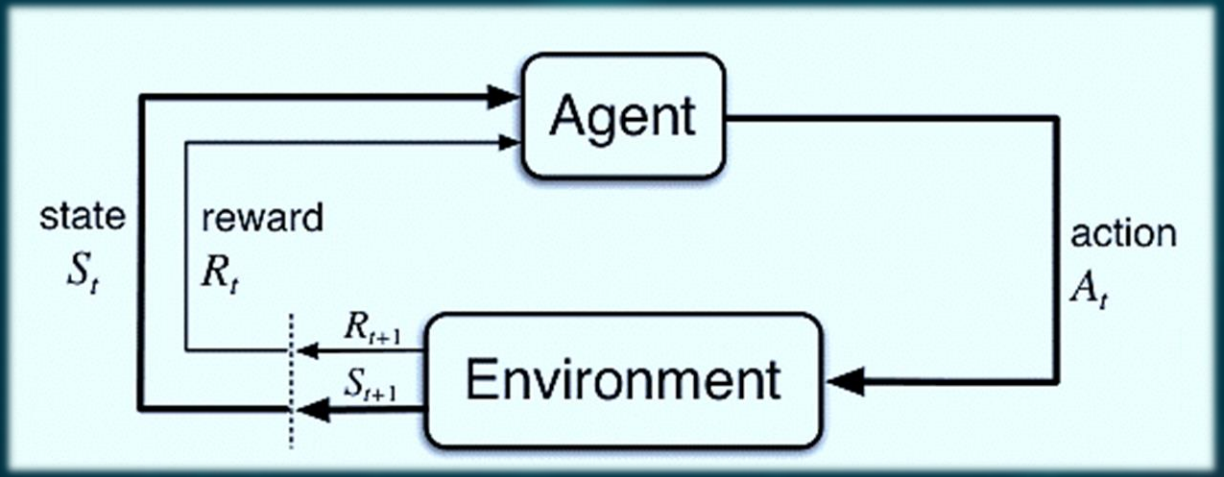
Ці дві характеристики — пошук методом проб і помилок і відкладена винагорода — є двома найважливішими відмінними рисами навчання з підкріпленням.

Навчання з підкріпленням

Навчання з підкріпленням, як і багато тем, назви яких закінчуються на «ing», як-от machine learning, є водночас проблемою, класом методів вирішення, які добре працюють із проблемою, і галуззю, яка вивчає цю проблему та методи її вирішення.

Зручно використовувати одну назву для всіх трьох речей, але в той же час важливо зберегти три концептуально розділеними.

Основна ідея полягає в тому, щоб просто охопити найважливіші аспекти реальної проблеми, з якою стикається агент, що навчається, який протягом часу взаємодіє зі своїм середовищем для досягнення мети.

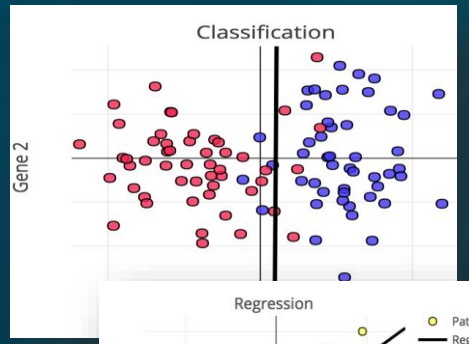


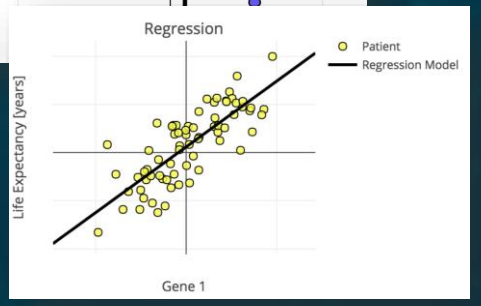
Агент повинен певною мірою сприймати стан свого оточення та вживати дій, які впливають на цей стан.

Агент також повинен мати ціль або цілі, пов'язані зі станом навколишнього середовища.

Навчання з підкріпленням відрізняється від навчання з вчителем, типу навчання, яке вивчається в більшості сучасних досліджень у галузі машинного навчання.

Контрольоване навчання — це навчання з навчального набору позначених прикладів, де кожен приклад є описом ситуації разом із специфікацією — міткою — правильної дії, яку система має виконати щодо цієї ситуації, що часто полягає в ідентифікації категорії, до якої належить ситуація.

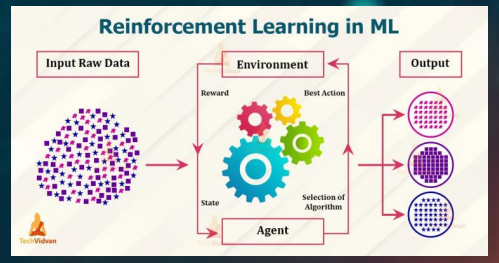




Мета такого навчання полягає в тому, щоб система екстраполювала або узагальнювала свої відповіді, щоб вона діяла правильно в ситуаціях, яких немає в навчальному наборі. Це важливий вид навчання, але сам по собі він недостатній для навчання на основі взаємодії.

vs

В інтерактивних задачах часто непрактично отримати приклади бажаної поведінки, які є водночас правильними та репрезентативними для всіх ситуацій, у яких агент повинен діяти.

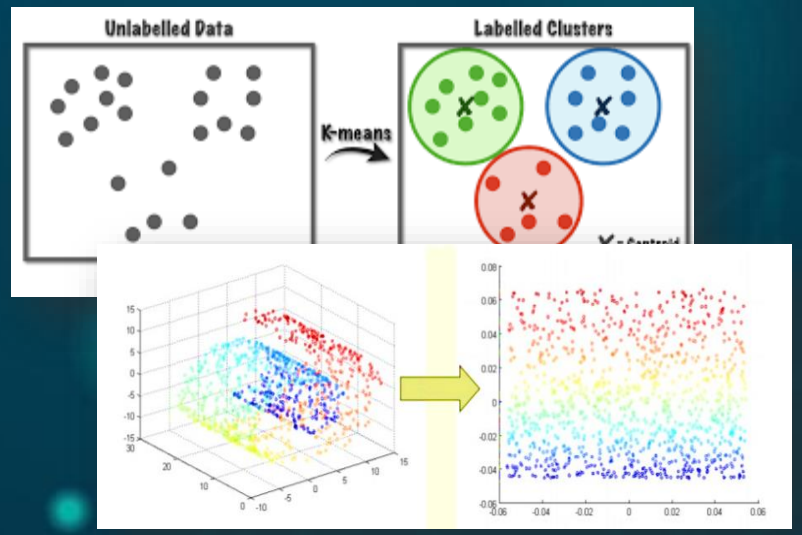


На незвіданій території, де можна було б очікувати, що навчання принесе найбільшу користь, агент повинен мати можливість вчитися на власному досвіді.

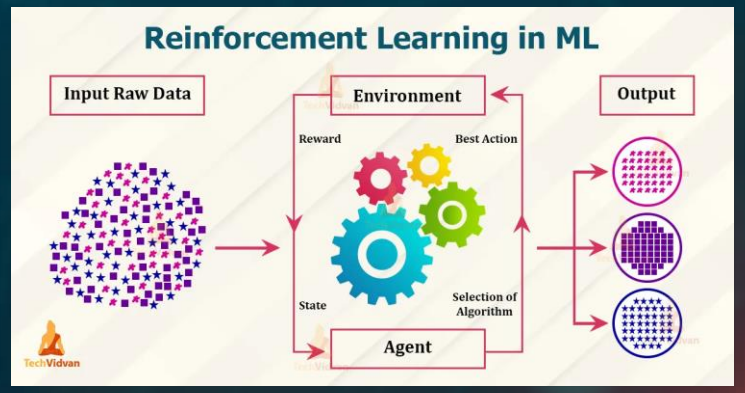
Навчання з підкріпленням також відрізняється від неконтрольованого навчання, яке зазвичай стосується пошуку структури, прихованої в колекціях немаркованих даних.

Навчання з підкріпленням намагається максимізувати сигнал винагороди замість того, щоб намагатися знайти приховану структуру.

Розкриття структури в досвіді агента, безумовно, може бути корисним у навчанні з підкріпленням, але саме по собі не вирішує проблеми навчання з підкріпленням щодо максимізації сигналу винагороди.



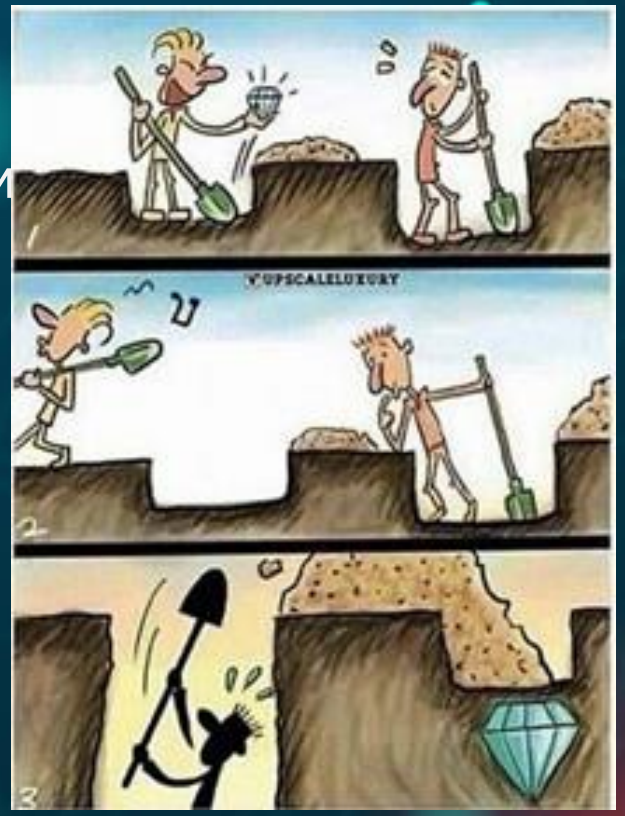
vs



Exploration & Exploitation

Одним із викликів, які виникають у навчанні з підкріпленням є компроміс між дослідженням та використанням (exploration and exploitation).

Щоб отримати значну винагороду, агент навчання з підкріпленням повинен віддавати перевагу діям, які він пробував у минулому та які виявилися ефективними для отримання винагороди. Але щоб виявити такі дії, він повинен спробувати дії, які він раніше не вибирав.



Агент має використати те, що він уже пережив, щоб отримати винагороду, але він також має дослідити, щоб зробити кращий вибір дій у майбутньому.

Дилема полягає в тому, що ані дослідження , ані використання не можуть здійснюватися виключно без невдачі у виконанні завдання. Агент повинен спробувати різноманітні дії та поступово віддавати перевагу тим, які здаються найкращими.

У стохастичному завданні кожну дію потрібно спробувати багато разів, щоб отримати надійну оцінку очікуваної винагороди.

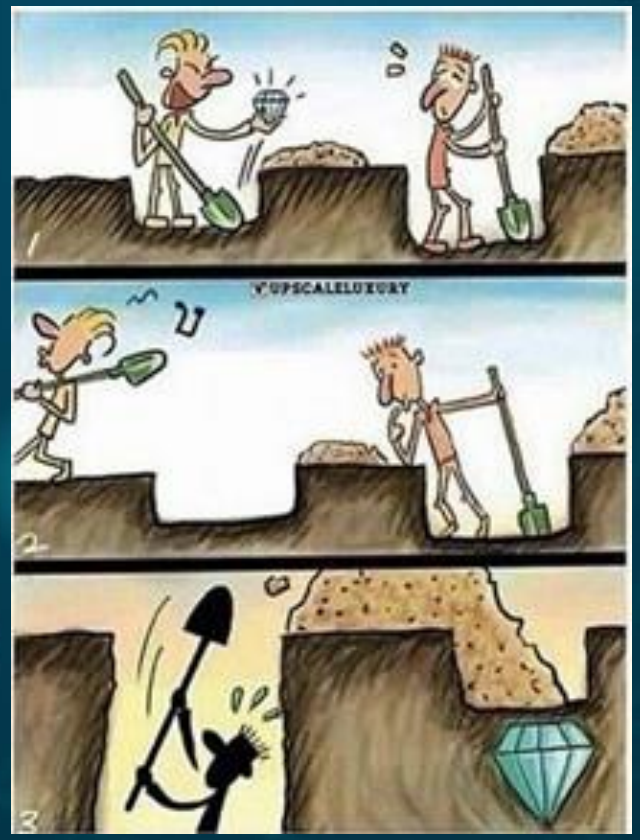
Елементи навчання з підкріпленням. Policy

Окрім агента та середовища, можна виділити чотири основні елементи системи навчання з підкріпленням: політику, сигнал винагороди, функцію цінності та, за бажанням, модель середовища.

Політика (**policy**) визначає спосіб поведінки агента в певний час. Грубо кажучи, політика – це відображення сприйнятого стану навколишнього середовища на дії, які необхідно вжити, перебуваючи в цих станах.

Це відповідає тому, що можна назвати набором правил або асоціацій стимул-реакція. У деяких випадках політика може бути простою функцією або таблицею пошуку, тоді як в інших вона може включати великі обчислення, такі як процес пошуку.

Політика є ядром агента навчання з підкріпленням у тому сенсі, що її однієї достатньо для визначення поведінки. Загалом політики можуть бути стохастичними, вказуючи ймовірності для кожної дії.



Ця дія називається жадібною дією, а політика — жадібною політикою (**greedy policy**).

Однак у цій ситуації політика жадібності зазнає невдачі, оскільки там, де копали спочатку, закопано більший діамант.

Елементи навчання з підкріпленням. Reward signal

Сигнал винагороди (**reward signal**) визначає мету проблеми навчання з підкріпленням. На кожному кроці часу середовище надсилає агенту навчання з підкріпленням єдине число, яке називається винагородою.

Єдина мета агента — максимізувати загальну винагороду, яку він отримує в довгостроковій перспективі. Сигнал винагороди, таким чином, визначає, які є хороші та погані події для агента.

Вони є безпосередніми і визначальними рисами проблеми, з якою стикається агент.

Сигнал винагороди є основою для зміни політики; якщо дія, вибрана політикою, супроводжується низькою винагородою, тоді політику можна змінити, щоб вибрати іншу дію в цій ситуації в майбутньому.

Загалом сигнали винагороди можуть бути стохастичними функціями стану навколишнього середовища та вжитих дій.

Елементи навчання з підкріпленням. Value function

Тоді як сигнал винагороди вказує на те, що добре в безпосередньому сенсі, **функція цінності** (**value function**) визначає, що добре в довгостроковій перспективі.

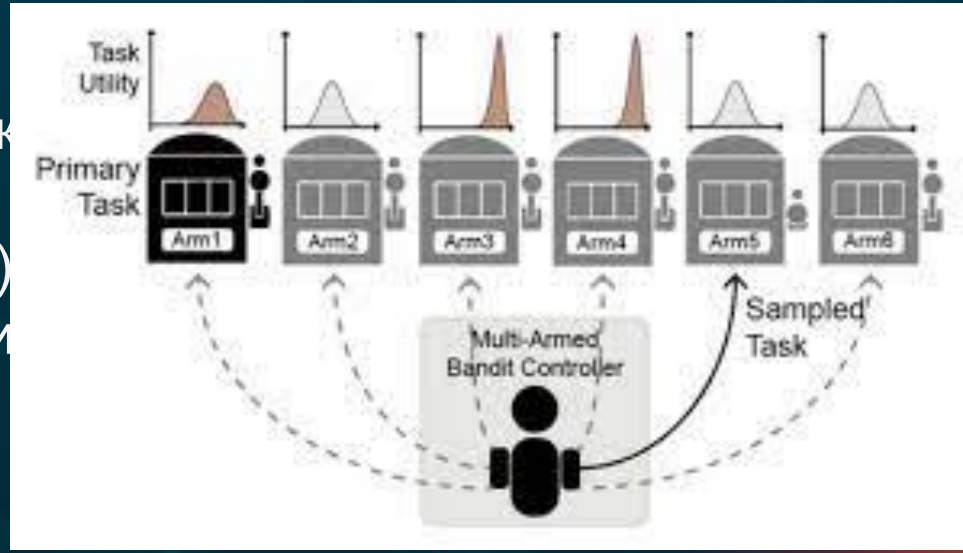
Грубо кажучи, цінність стану — це загальна сума винагороди, яку агент може очікувати накопичити в майбутньому, починаючи з цього стану. У той час як винагороди визначають негайну бажаність станів навколишнього середовища, цінності вказують на довгострокову бажаність станів після врахування станів, які, ймовірно, настануть, і винагород, доступних у тих станах.

Наприклад, стан може завжди приносити низьку негайну винагороду, але все ще мати високу цінність, оскільки за ним регулярно йдуть інші стани, які приносять високі винагороди. Або може бути навпаки.

MAB problem

У теорії ймовірностей та машинному навчанні **задача багаторукого бандита** (**Multi-Armed Bandit** яку іноді називають ***задачею K-або N-рукого* бандита**) — це задача в якій особа, яка приймає рішення, ітеративно вибирає один з декількох фіксованих варіантів, коли властивості кожного вибору відомі лише частково на момент розподілу і можуть стати краще зрозумілими з плином часу або шляхом розподілу ресурсів для реалізації варіанту.

Це класична задача навчання з підкріпленням, яка є прикладом дилеми балансу між експлуатацією та розвідк
Назва походить від уявного гравця на низці ігрових автоматів (їх часто називають «однорукими бандитами»)
який має вирішити, яку стратегію краще використовувати



Розглянемо наступну навчальну задачу. Ви неодноразово стикаєтеся з вибором серед **k** різних варіантів чи дій. Після кожного вибору ви отримуєте числову винагороду, вибрану зі стаціонарного розподілу ймовірностей, що залежить від вибраної дії. Ваша мета — максимізувати очікувану загальну винагороду протягом певного періоду часу, наприклад, вибравши понад 1000 дій або часових кроків.

Це оригінальне форулювання проблеми k-armed bandit, названої так за аналогією з гральним автоматом або «одноруким бандитом», за винятком того, що вона має k важелів замість одного.

Кожен вибір дії схожий на дію одного з важелів ігрового автомата, а винагорода — це винагорода за виграш джекпоту. Через повторний вибір дій потрібно максимізувати свій виграші, зосередивши свої дії на найкращих важелях.

Інша аналогія полягає в тому, що лікар вибирає між експериментальними методами лікування важкохворих пацієнтів. Кожна дія — це вибір лікування, а кожна нагорода — це благополуччя пацієнта.

У проблемі k-рукого бандита кожна з **k** дій має очікувану або середню винагороду, якщо цю дію вибрано; назвемо це **цінністю** (value) цієї дії.

Дію, обрану на кроці часу t, позначають як A_t, а відповідну винагороду як R_t. Тоді цінність довільної дії **a**, позначене $q_*(a)$, є очікуваною винагородою, якщо вибрано a:

$$q_*(a) \doteq \mathrm{E}\left[R_t | A_t = a\right]$$

Якби була відома цінність кожної дії, тоді було б тривіально розв'язати проблему к-рукого бандита : завжди вибирали б дію з найвищою цінністю.

Припускаємо, що цінність дії не відома напевно, хоча можуть бути якісь оцінки. Позначимо оцінену цінність дії a на кроці часу t як $Q_t(a)$. Бажано, щоб значення $Q_t(a)$ було близько до $q_*(a)$.

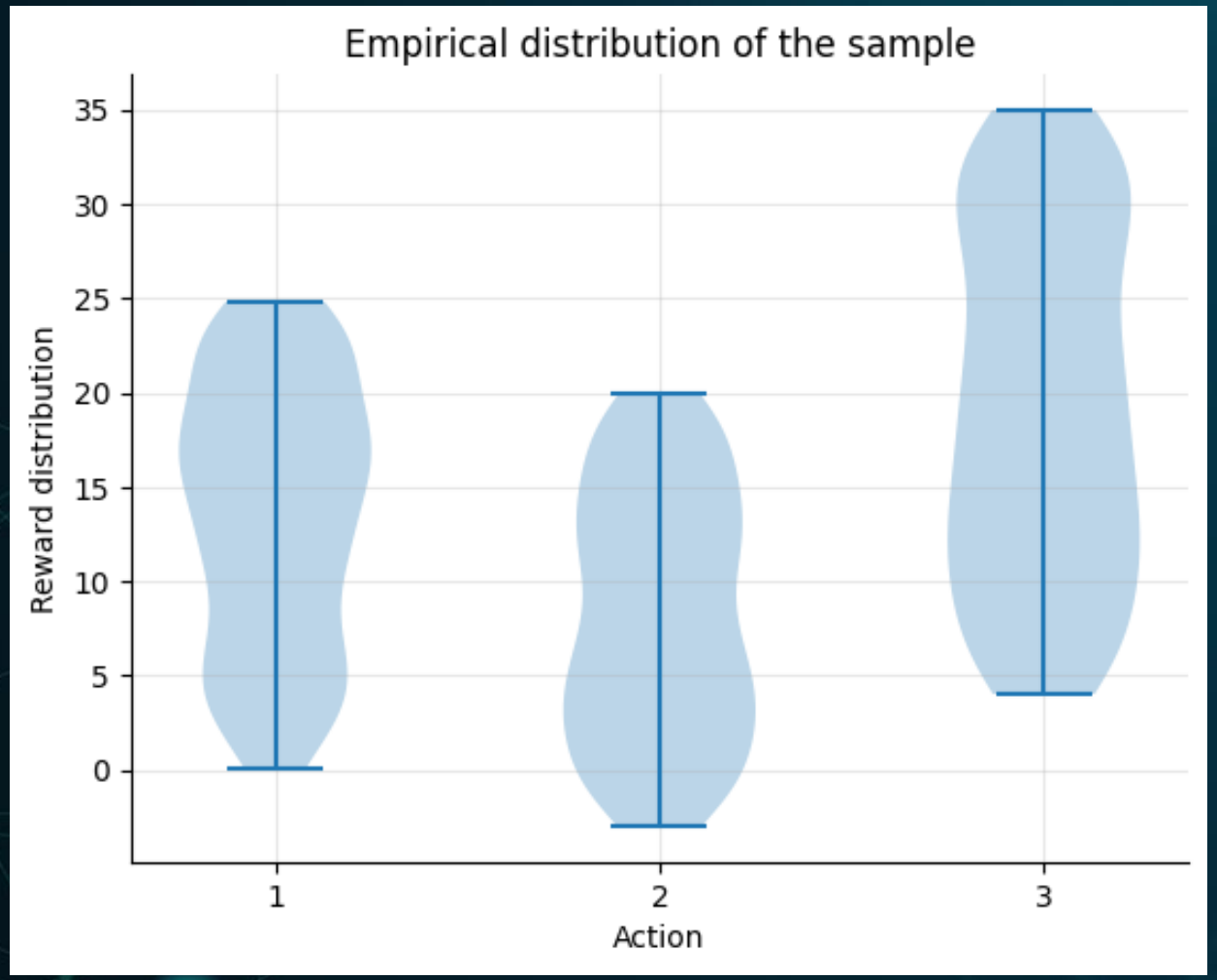
Якщо ви у продовж часу оцінюєте цінність дій, то на будь-якому часовому кроці існує принаймні одна дія, оцінена вартість якої є найбільшою. Їх називають **жадібними діями**.

Коли обирають одну з цих дій, то це означає «використання» поточних знань про значення дій. Якщо натомість обирають одну з нежадібних дій, це означає «досліджувати», оскільки це дає змогу покращити оцінку цінності нежадібної дії. Експлуатація — це правильна річ, щоб максимізувати очікувану винагороду за один крок, але дослідження може принести більшу загальну винагороду в довгостроковій перспективі.

Наприклад, припустимо, що цінність жадібної дії відома з упевненістю, тоді як кілька інших дій оцінюються як майже такі ж хороші, але зі значною невизначеністю. Невизначеність така, що принаймні одна з цих інших дій, ймовірно, насправді є кращою, ніж жадібна дія, але невідомо яка.

Якщо є багато часових кроків, на яких потрібно зробити вибір дій, тоді, можливо, краще дослідити нежадібні дії та виявити, які з них кращі за жадібні.

Винагорода нижча в короткостроковій перспективі під час дослідження, але вища в довгостроковій перспективі, тому що після того, як виявили кращі дії, можна використовувати їх багато разів. Оскільки неможливо одночасно досліджувати та використовувати за допомогою будь-якої однієї дії, говорять про «дилему» між дослідженням та використанням.



Одним із природних способів оцінити це є усереднення фактично отриманих винагород:

$$Q_t(a) \doteq \frac{\sum_{i=1}^{t-1} R_i * \mathbb{I}(A_i = a)}{\sum_{i=1}^{t-1} \mathbb{I}(A_i = a)}$$

сума винагород, отриманих до t кількість разів a, взятих до t

Де $\mathbb{I}(A_i = a)$ предикат позначає випадкову змінну, яка дорівнює 1, якщо предикат істинний, і 0, якщо ні.

Якщо знаменник дорівнює нулю, замість цього визначають $Q_t(a)$ як деяке значення за замовчуванням, наприклад 0.

Коли знаменник прагне до нескінченності $Q_t(a)$ збігається до $q_*(a)$.

Найпростішим правилом вибору дій є вибір однієї з дій з найвищою оцінкою, тобто однієї з жадібних дій, як визначено раніше.

Якщо є більше ніж одна жадібна дія, то вибір робиться серед них якимось довільним способом, можливо, випадковим чином. Запишемо цей метод вибору жадібних дій як

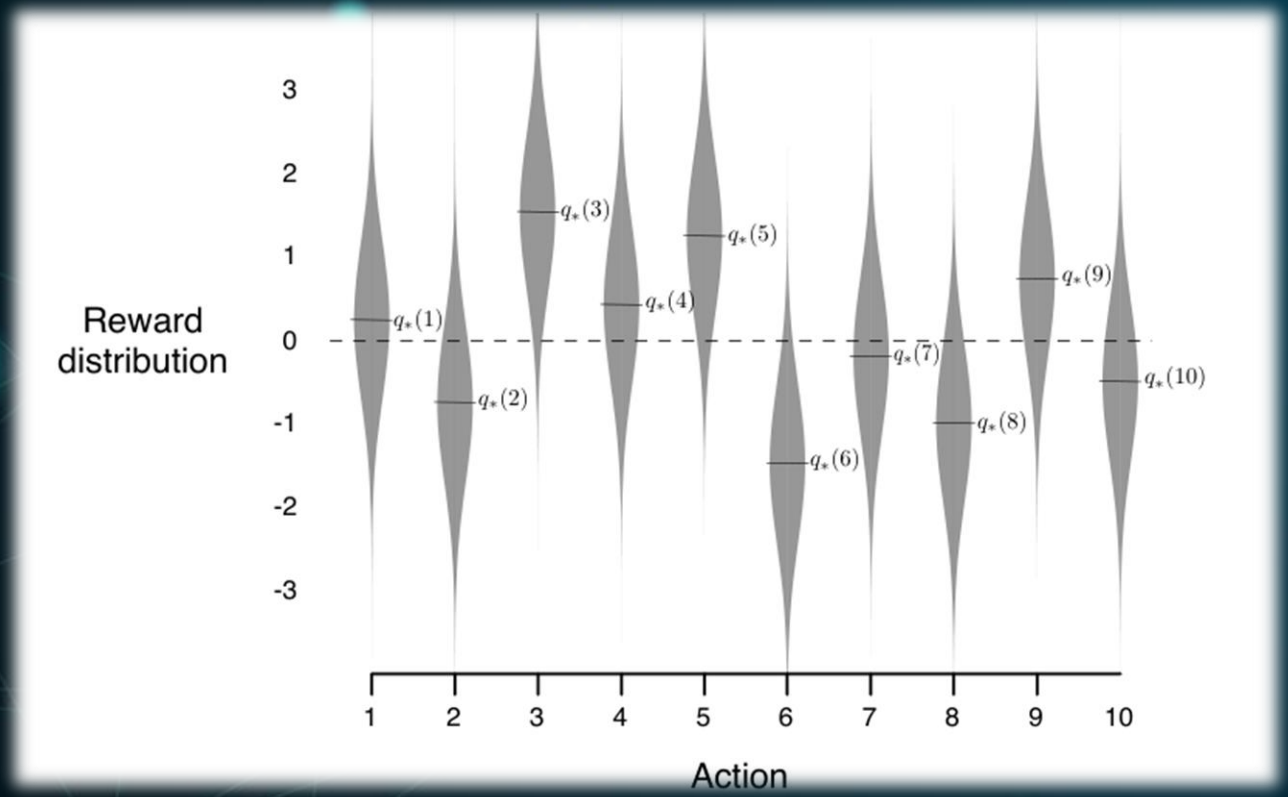
$$A_t \doteq argmax\, Q_t(a)$$

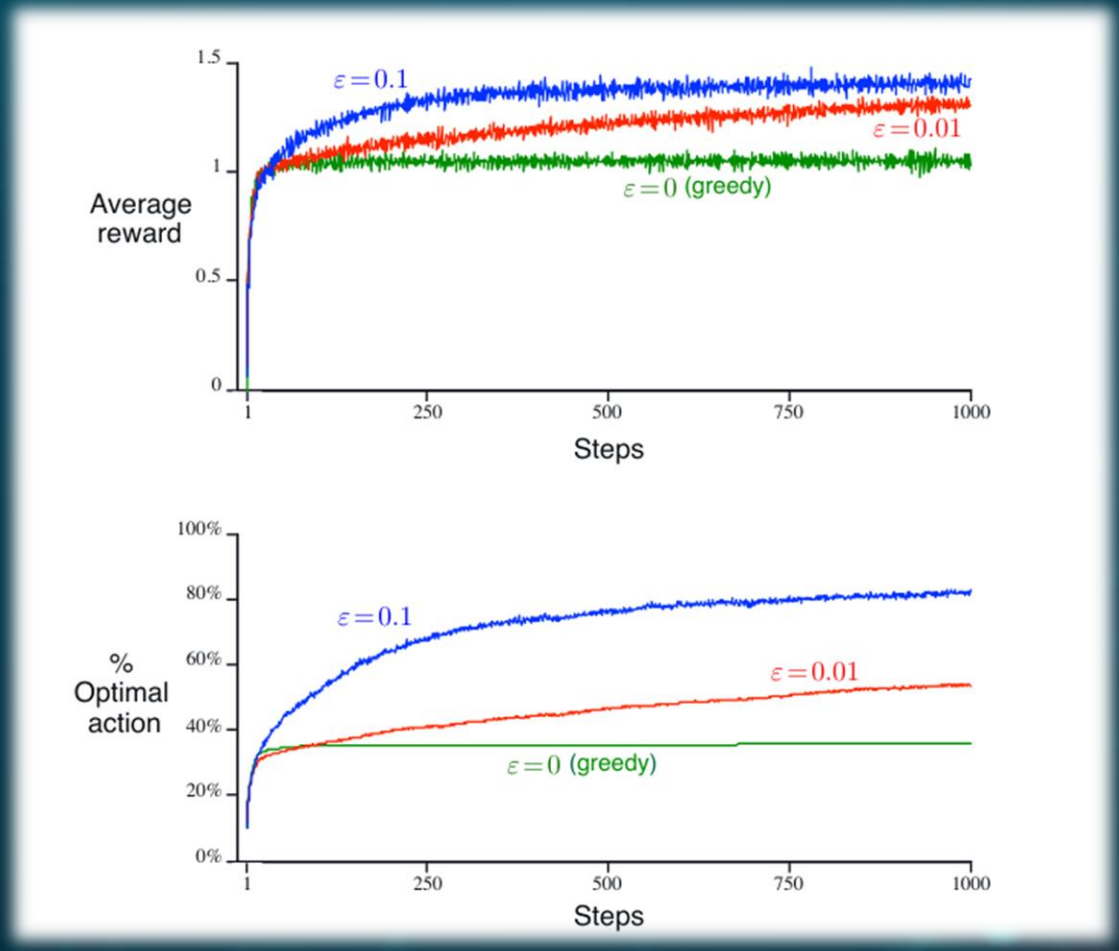
Жадібний вибір дій завжди використовує поточні знання для максимізації негайної винагороди; він взагалі не витрачає часу на вибір сумнівних дій, щоб побачити, чи дійсно вони можуть бути кращими.

Проста альтернатива полягає в тому, щоб більшу частину часу поводитися жадібно, але час від часу, скажімо, "з невеликою ймовірністю" ε, натомість вибирати випадковим чином серед усіх дій з рівною ймовірністю, незалежно від оцінок цінності дій.

Ці методи, що використовують таке майже жадібне правило вибору дії називають ε-жадібні методи. Перевага цих методів полягає в тому, що в межах, коли кількість кроків збільшується, кожна дія буде відбиратися нескінченну кількість разів, таким чином гарантуючи, що всі $Q_t(a)$ збігаються до $q_*(a)$. Це, звичайно, означає, що ймовірність вибору оптимальної дії збігається до більш ніж $1 - \varepsilon$, тобто до майже напевно.

Приклад задачі про бандита з тестового стенда з 10 руками.





Середня продуктивність методів ε-greedy action-value" на тестовому стенді з 10 групами.
Ці дані є середніми за 2000 тестів із різними проблемами бандитів. Усі методи використовували вибіркові середні значення як оцінку цінності дії.

Reinforcement Learning: An Introduction second edition
by Richard S. Sutton and Andrew G. Barto

Додаткові формули для оновлення середніх значень із невеликими постійними обчисленнями, необхідними для обробки кожної нової винагороди.

Враховуючи Q_n і n-ту винагороду, R_n, нове середнє значення всіх n винагород можна обчислити за допомогою

$$Q_{n+1} = Q_n + \frac{1}{n}[R_n - Q_n]$$

A simple bandit algorithm

Initialize, for $a = 1$ to k:

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Loop forever:

$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

$R \leftarrow bandit(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

Reinforcement Learning: An Introduction
second edition
by Richard S. Sutton and Andrew G. Barto

Бібліотека PyGame

Використовується бібліотека Python PyGame для створення відеоігор.
Ця бібліотека включає кілька модулів для відтворення звуку, графіки, обробки подій тощо. за допомогою Python.

За замовчуванням Python не постачається з PyGame як вбудованою бібліотекою. Встановити Pygame можна за допомогою команди:

```
pip install pygame
```

Для початку роботи імпортувати модуль pygame та викликати функцію init() для підготовки модулів pygame до роботи:

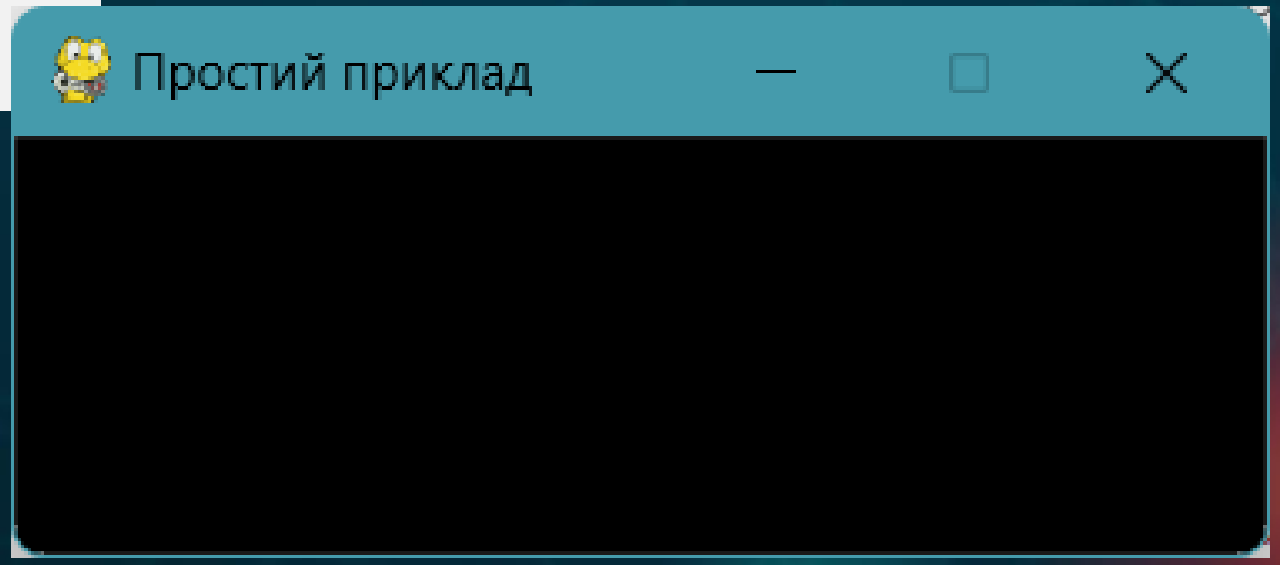
```
import pygame
from pygame.locals import *
pygame.init()
```

Простий приклад

```python
import pygame, sys
from pygame.locals import *

pygame.init()
DISPLAYSURF = pygame.display.set_mode((300, 100))
pygame.display.set_caption('Простий приклад')
while True: # main game loop
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
    pygame.display.update()
```

Повертає pygame.Surface об’єкт (що представляє видиму частину вікна), задає ширину та висоту вікна в пікселях. (300, 100) створить вікно шириною 300 пікселів і висотою 100 пікселів.



Ігрові цикли та ігрові стани

- Кожна гра використовує ігровий цикл для керування ігровим процесом. Ігровий цикл робить чотири дуже важливі речі:
  - Обробляє дані користувача
  - Оновлює стан усіх ігрових об'єктів
  - Оновлює дисплей і аудіовихід
  - Зберігає швидкість гри

Кожного разу, коли користувач виконує одну з кількох дій, наприклад натискання клавіші клавіатури або переміщення миші у вікні програми, бібліотека Pygame створює об’єкт pygame.event.Event для запису цієї «події». (Це тип об’єкта під назвою Event, який існує в модулі події, який сам знаходиться в модулі pygame.)

Щоб дізнатися, які події відбулися, є функція pygame.event.get(), яка повертає список об’єктів Event. У списку об’єктів Event буде кожна подія, яка сталася з моменту останнього виклику функції pygame.event.get(). (Або з моменту запуску програми.)

```python
import pygame, sys
from pygame.locals import *

pygame.init()
DISPLAYSURF = pygame.display.set_mode((300, 100))
pygame.display.set_caption('Простий приклад')
while True: # main game loop
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
    pygame.display.update()
```

Цикл for, який виконуватиме ітерацію по списку об’єктів Event, які повертає pygame.event.get().

На кожній ітерації циклу for змінній з іменем **event** буде присвоєно значення наступного об’єкта події в цьому списку.

Список об’єктів Event, які повертає pygame.event.get(), буде в тому порядку, у якому відбулися події.

Якщо користувач натиснув кнопку миші, а потім натиснув клавішу клавіатури, об’єкт Event для клацання миші буде першим у списку, а об’єкт Event для натискання клавіатури буде другим. Якщо жодних подій не сталося, тоді pygame.event.get() поверне порожній список.

Об’єкти подій мають змінну (також звану атрибутами або властивостями) з назвою type, яка повідомляє, яку подію представляє об’єкт.
Pygame має постійну змінну для кожного з можливих типів у модулях pygame.locals.

```
import pygame, sys
from pygame.locals import *

pygame.init()
DISPLAYSURF = pygame.display.set_mode((300, 100))
pygame.display.set_caption('Простий приклад')
while True: # main game loop
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
    pygame.display.update()
```

Перевіряє, чи дорівнює тип об’єкта Event константі QUIT.

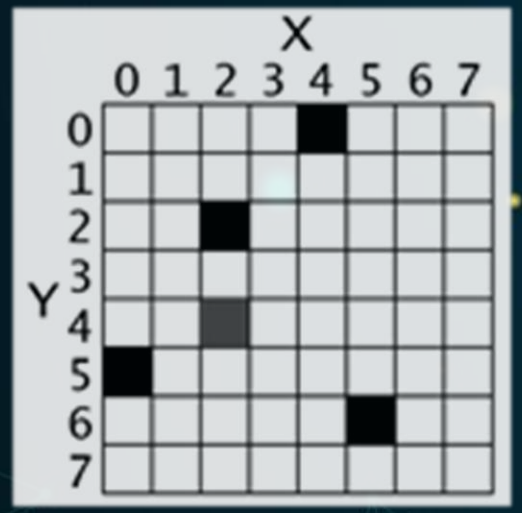
(Оскільки використовували форму оператора імпорту from pygame.locals import *, то потрібно лише ввести QUIT замість pygame.locals.QUIT).

Якщо об’єкт Event є подією виходу, то викликаються функції pygame.quit() і sys.exit().

Функція pygame.quit() це щось протилежне функції pygame.init(): вона запускає код, який дезактивує бібліотеку Pygame. Слід завжди викликати pygame.quit() перед викликом sys.exit() для завершення програми.

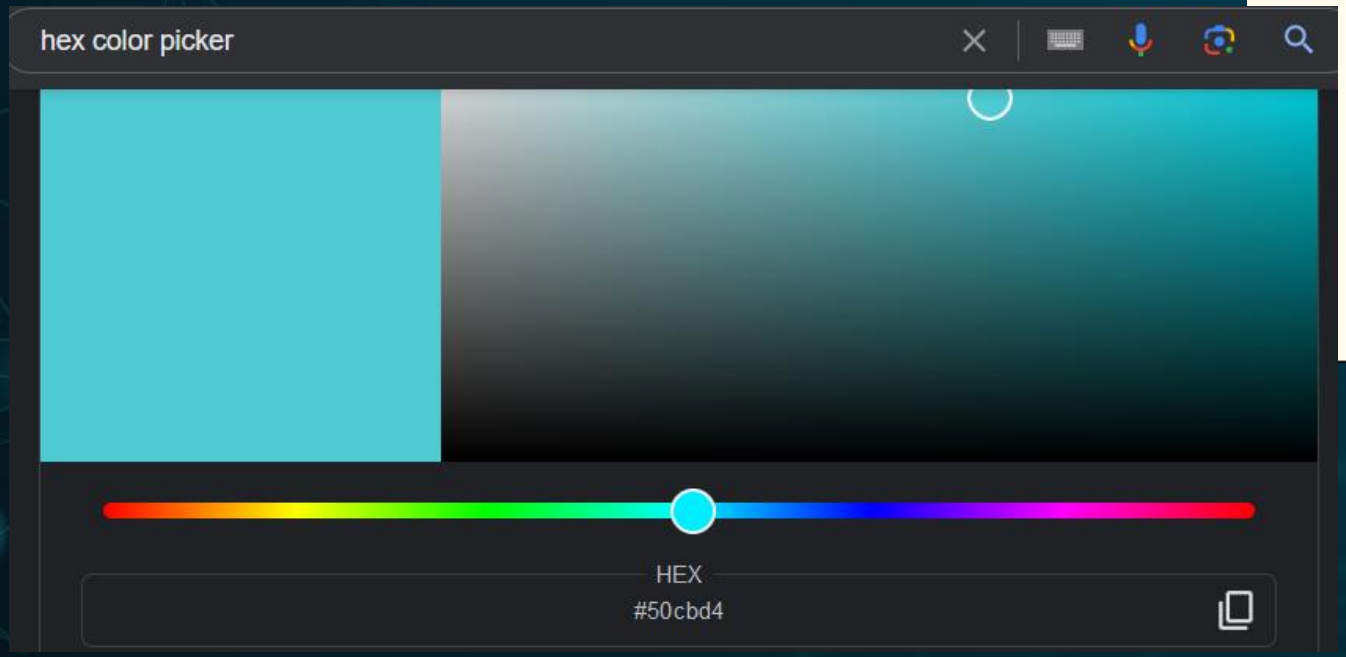
Зазвичай це не має особливого значення, оскільки Python все одно закриває його, коли програма завершує роботу. Але в IDLE є помилка, через яку IDLE зависає, якщо програма Pygame завершується до виклику pygame.quit().

Координати пікселів та кольори



XY координати:
(4, 0), (2, 2), (0, 5), та (5, 6)

Color	RGB Values
Aqua	(0, 255, 255)
Black	(0, 0, 0)
Blue	(0, 0, 255)
Fuchsia	(255, 0, 255)
Gray	(128, 128, 128)
Green	(0, 128, 0)
Lime	(0, 255, 0)
Maroon	(128, 0, 0)
Navy Blue	(0, 0, 128)
Olive	(128, 128, 0)
Purple	(128, 0, 128)
Red	(255, 0, 0)
Silver	(192, 192, 192)
Teal	(0, 128, 128)
White	(255, 255, 255)
Yellow	(255, 255, 0)



Формальна проблема скінченних марковських процесів прийняття рішень, або скінченних MDP (Markov decision processes)

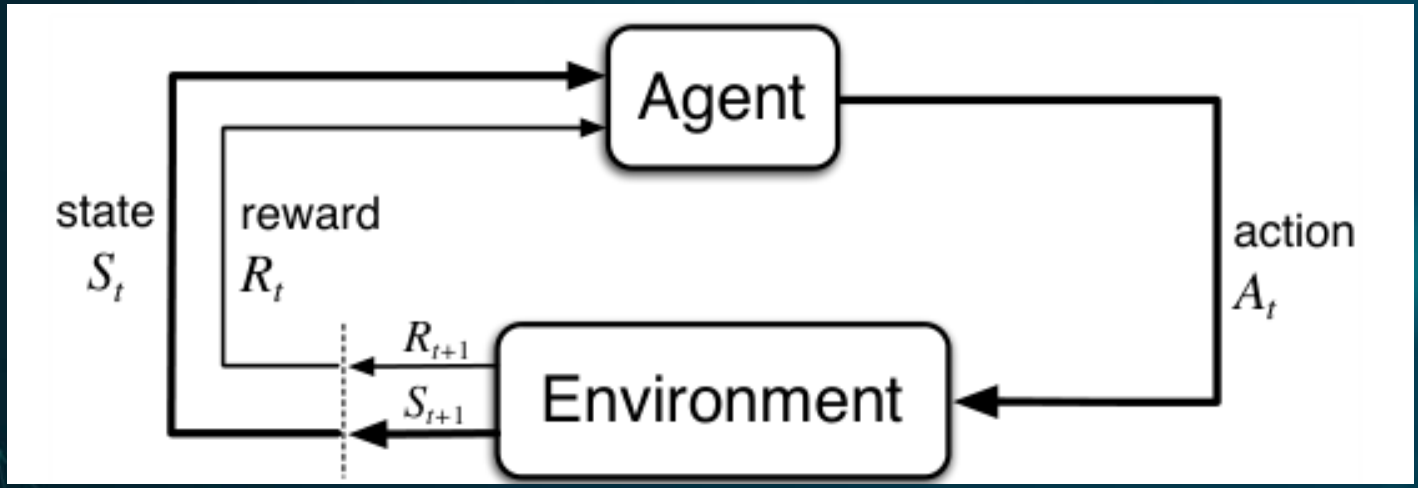
Ця проблема передбачає оціночний зворотний зв’язок, як у k-armed task, а також асоціативний аспект — вибір різних дій у різних ситуаціях. MDP — це класична формалізація послідовного прийняття рішень, коли дії впливають не лише на негайні винагороди, але й на наступні ситуації чи стани, а через це на майбутні винагороди.

Таким чином, MDP передбачає відкладену винагороду та необхідність компромісу між негайною та відкладеною винагородою. Тоді як у k-armed проблемах оцінювали значення $q_*(a)$ кожної дії **a**, у MDP оцінюють значення $\mathbf{q_*(s, a)}$ кожної дії **a** в кожному стані **s**, або оцінюють значення $\mathbf{v_*(s)}$ кожного стану при виборі оптимальної дії.

Ці величини, що залежать від стану, важливі для точного присвоєння довгострокових наслідків індивідуальним виборам дій.

MDP є математично ідеалізованою формою проблеми навчання з підкріпленням, для якої можна зробити точні теоретичні твердження.

Взаємодія агент-середовище в марковському процесі прийняття рішень



Більш конкретно, агент і середовище взаємодіють на кожному з послідовності дискретних часових кроків, t = 0, 1, 2, 3, ...

На кожному кроці часу **t** агент отримує певне представлення **стану** середовища, $S_t \in S$, і на цій основі вибирає дію $A_t \in A(s)$.

На один крок пізніше, частково як наслідок за виконаної **дії** агент отримує числову **винагороду**, $R_{t+1} \in R \subset \mathbb{R}$, і опиняється в новому стані, S_{t+1}.

MDP і агент разом таким чином створюють послідовність або **траєкторію**, яка починається так:S0, A0, R1, S1, A1, R2, S2, A2, R3, . . .

Мета агента полягає в тому, щоб максимізувати сукупну винагороду, яку він отримує в довгостроковій перспективі.

Якщо послідовність винагород, отриманих після кроку часу **t**, позначається як R_{t+1}, R_{t+2}, R_{t+3}, . . ., тоді прагнуть максимізувати очікувану віддачу, де віддача, позначена G_t, визначається як певна функція послідовності винагороди.
У найпростішому випадку віддача є сумою винагород:

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T \ , \quad (**)$$

де T - кінцевий крок часу. Цей підхід має сенс у додатках, у яких існує природне поняття кінцевого часового кроку, тобто коли взаємодія агента та середовища природним чином розбивається на підпослідовності, так звані **епізоди**, такі як раунд гри, подорожі лабіринтом або будь-яка повторна взаємодія. Кожен епізод закінчується в особливому стані, який називається кінцевим станом, після чого відбувається скидання до стандартного початкового стану або до зразка зі стандартного розподілу початкових станів.
Епізоди закінчуються по-різному, наприклад виграш або програш у грі, наступний епізод починається незалежно від того, як закінчився попередній. Таким чином, можна вважати, що всі епізоди закінчуються в одному термінальному стані, з різними винагородами за різні результати. Завдання з епізодами такого роду називаються **епізодичними**. В епізодичних завданнях іноді потрібно відрізнити набір усіх нетермінальних станів, позначений S, від набору всіх станів плюс термінальний стан, позначений S+. Час завершення, T, є випадковою величиною, яка зазвичай змінюється від епізоду до епізоду.

З іншого боку, у багатьох випадках взаємодія агент-оточення не розривається природним чином на ідентифіковані епізоди, а триває безперервно без обмежень.

Наприклад, це був би природний спосіб сформулювати поточне завдання керування процесом або застосувати до робота з тривалим терміном служби. Такі завдання називають **безперервними** завданнями.

Застосування формули (**) є проблематичним для безперервних завдань, оскільки останній часовий крок буде T = 1, а віддача, яку ми намагаємося максимізувати, сама по собі може бути нескінченною.

(Наприклад, припустімо, що агент отримує винагороду +1 на кожному часовому етапі.)

Додаткова концепція, це дисконтування. Відповідно до цього підходу агент намагається вибрати дії так, щоб сума дисконтованих винагород, які він отримує в майбутньому, була максимальною. Зокрема, він вибирає A_t, щоб максимізувати очікувану дисконтовану винагороду:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

де γ— параметр, $0 \leq \gamma \leq 1$, який називається ставкою дисконту.

Політики та функції цінностей

Майже всі алгоритми навчання з підкріпленням передбачають оцінку **функцій цінностей** — функцій станів (або пар стан–дія), які оцінюють, наскільки добре для агента перебувати в даному стані (або наскільки добре виконувати певну дію в даному стані).

Поняття «наскільки добре» тут визначається в термінах майбутніх винагород, які можна очікувати, або, якщо бути точним, в термінах очікуваної винагороди.

Звичайно, винагорода, яку агент може очікувати отримати в майбутньому, залежить від того, які дії він вживатиме. Відповідно, функції цінностей визначаються щодо конкретних способів дій, які називаються **політиками.**

Формально **політика** — це відображення від станів до ймовірностей вибору кожної можливої дії. Якщо агент дотримується політики π у момент часу t, тоді $\pi(a|s)$ є ймовірністю того, що $A_t = a$, якщо $S_t = s$. Як і p, π є звичайною функцією; «|» в середині $\pi(a|s)$ просто нагадує, що він визначає розподіл ймовірностей на $a \in A(s)$ для кожного $s \in S$. Методи навчання з підкріпленням визначають, як політика агента змінюється в результаті його досвіду.

Функція цінності стану s за політикою π, що позначається $v_\pi(s)$, є очікуваною винагородою, починаючи з **s** і дотримуючись політики **π** після цього. Для MDP ми можемо формально визначити $\mathbf{v}_\pi$ за допомогою

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right], \text{ for all } s \in \mathcal{S},$$

де $\mathbb{E}_\pi[\cdot]$ позначає очікуване значення випадкової змінної, враховуючи, що агент дотримується політики π, а t є будь-яким часовим кроком.

Значення термінального стану, якщо таке є, завжди дорівнює нулю.

Функцію v_π називають функцією цінності стану для політики π. Подібним чином визначають цінність виконання дії **a** у стані **s** згідно з політикою **π**, позначеною $\mathbf{q_\pi(s, a)}$, як очікувану винагороду, починаючи з **s**, виконуючи дію **a**, а потім дотримуючись політики **π** :

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s, A_t = a\right]$$

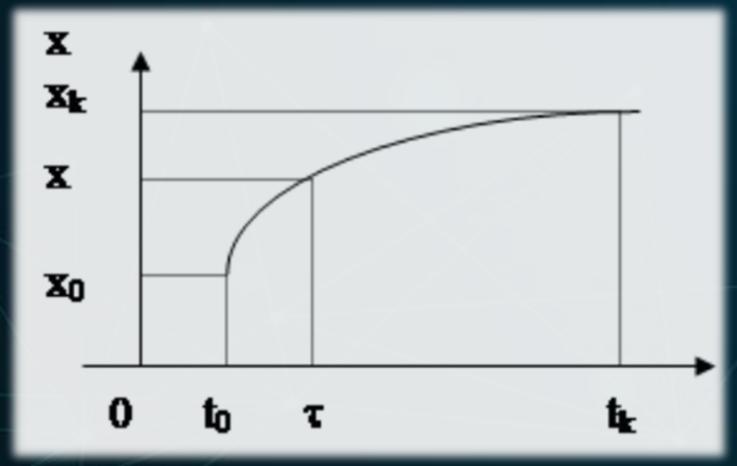
Рівняння Беллмана

Bellman equation

Метод динамічного програмування, запропонований на початку 1950-х років американським вченим Річардом Беллманом, передбачає вирішення загальної задачі оптимального управління.

Грунтується динамічне програмування на принципі оптимальності Беллмана, який можна сформулювати так:

Оптимальне управління визначається тільки кінцевою метою управління та станом системи в даний момент часу, незалежно від того, яким чином система прийшла в цей стан.



Тобто якщо траєкторія $x = f(t), t = \overline{[t_0, t_k]}$ є оптимальною, то будь-яка її частина $x = f(t), t = [\tau, t_k]$, що приводить до кінцевого стану X_k є також оптимальною траєкторією.

Під терміном “динамічне” він мав на увазі, що проблема оптимального управління, яку він вирішував цим методом є залежною від часу, тобто вона постійно змінюється.

Рівняння Беллмана для v_π

Фундаментальна властивість функцій цінності, які використовуються в навчанні з підкріпленням і динамічному програмуванні, полягає в тому, що вони задовольняють рекурсивні відносини, подібні до тих, що описані для очікуваної винагороди

$$
\begin{aligned}
G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\
&= R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots\big) \\
&= R_{t+1} + \gamma G_{t+1}
\end{aligned}
$$

Для будь-якої політики π і будь-якого стану **s**, наступна умова узгодженості виконується між цінністю стану **s** і цінністю його можливих наступних станів:

$$
\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\
&= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r \mid s, a) \Big[r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s']\Big] \\
&= \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) \Big[r + \gamma v_\pi(s')\Big], \quad \text{for all } s \in \mathcal{S},
\end{aligned}
$$

Рівняння Беллмана для v_π

(***)

Вирішення завдання навчання з підкріпленням означає, простими словами, пошук політики, яка забезпечує значну винагороду в довгостроковій перспективі.

Для скінченних MDP можна точно визначити оптимальну політику наступним чином. Функції цінності визначають часткове впорядкування політик.

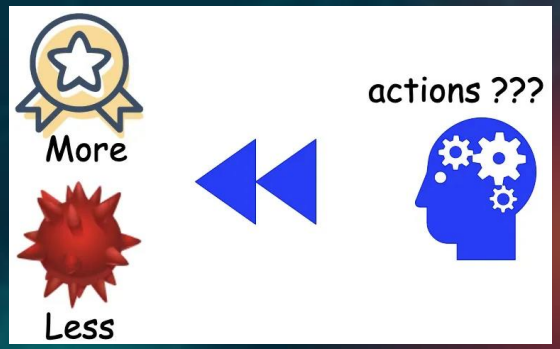
Політика $\boldsymbol{\pi}$ вважається кращою або рівною політиці $\boldsymbol{\pi}'$, якщо її очікувана винагорода більша або дорівнює винагороді при $\boldsymbol{\pi}'$ для всіх станів.

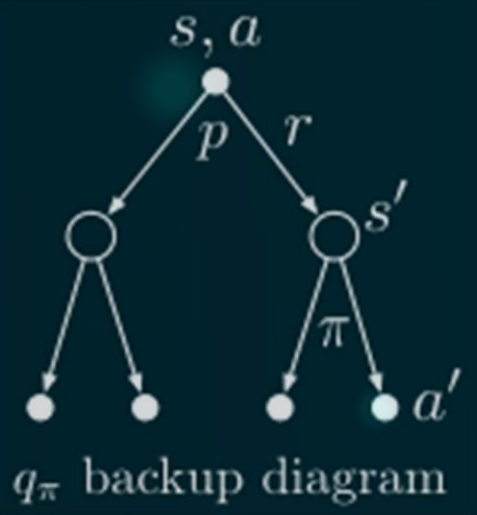
Іншими словами, $\boldsymbol{\pi} \geq \boldsymbol{\pi}'$ тоді і тільки тоді, коли $\boldsymbol{v}_{\boldsymbol{\pi}}(\boldsymbol{s}) \geq \boldsymbol{v}_{\boldsymbol{\pi}'}(\boldsymbol{s})$ для всіх $s \in S$.

Завжди існує принаймні одна політика, яка є кращою або рівною всім іншим політикам. Це **оптимальна політика**. Хоча їх може бути більше ніж одна, позначають всі оптимальні політики $\boldsymbol{\pi}_*$.

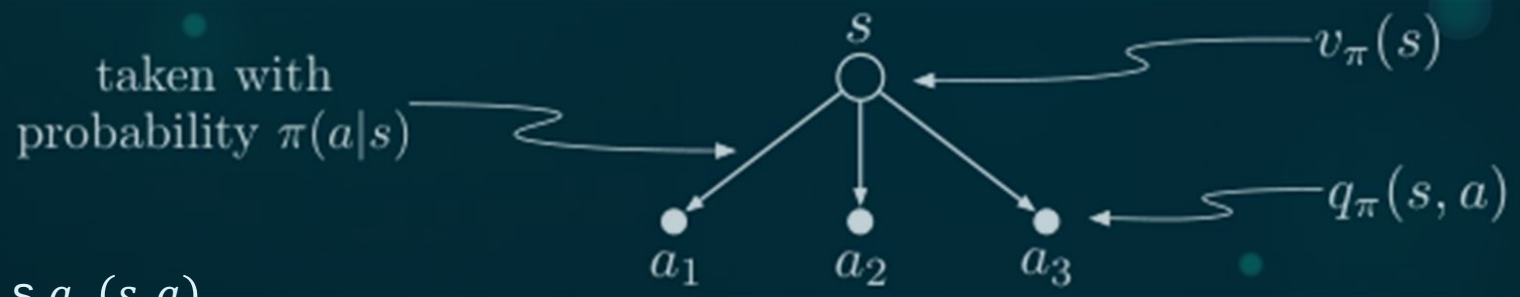
Вони мають однакову функцію цінності станів, яка називається **оптимальною функцією цінності станів**, позначається $\boldsymbol{v}_*$ і визначається як

$$v_*(s) \doteq \max_{\pi} v_\pi(s), \qquad \forall s \in S$$





Цінність дії a у стані s $q_\pi(s,a)$ визначається через цінність можливих наступних пар дія-стан $q_\pi(s',a')$.



Цінність стану залежить від цінностей дій, можливих у цьому стані, і від того, з якою ймовірністю буде обрано кожну дію відповідно до поточної політики.

Оптимальні політики також мають однакову оптимальну функцію дії-цінності, позначену $\boldsymbol{q}_*$ і визначену як

$$q_*(s,a) \doteq \max_\pi q_\pi(s,a), \qquad \forall s \in S, \forall a \in A$$

Для пари стан–дія **(s, a)** ця функція дає очікувану винагороду від виконання дії **a** у стані **s** і подальшого дотримання оптимальної політики.

Таким чином, можна записати q_* через v_* наступним чином:

$$q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$

Рівняння (оптимальності) Беллмана

Bellman optimality equation

Оскільки v_* є функцією цінності для політики, вона повинна задовольняти умову самоузгодженості, задану рівнянням Беллмана для значень стану (***). Проте, оскільки це функція оптимального значення, умова узгодженості v_* може бути записана в спеціальній формі без посилання на будь-яку конкретну політику.

Це рівняння Беллмана для v_*, або рівняння оптимальності Беллмана. Інтуїтивно зрозуміло, що рівняння оптимальності Беллмана виражає той факт, що цінність стану за оптимальної політики має дорівнювати очікуваній винагороді для найкращої дії з цього стану:

$$
\begin{aligned}
v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s,a) \\
&= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\
&= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\
&= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma v_*(s')\big].
\end{aligned}
$$

$$
\begin{aligned}
q_*(s,a) &= \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1},a') \;\Big|\; S_t = s, A_t = a\Big] \\
&= \sum_{s',r} p(s',r \mid s,a)\Big[r + \gamma \max_{a'} q_*(s',a')\Big].
\end{aligned}
$$

$$
\begin{aligned}
v_{k+1}(s) &\doteq \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s',r \mid s,a)\Big[r + \gamma v_k(s')\Big],
\end{aligned}
$$

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma V(s')\big]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

$$
\begin{aligned}
\pi'(s) &\doteq \arg\max_a q_\pi(s,a) \\
&= \arg\max_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \arg\max_a \sum_{s',r} p(s',r \mid s,a)\Big[r + \gamma v_\pi(s')\Big],
\end{aligned}
$$

$$
\begin{aligned}
v_{\pi'}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \max_a \sum_{s',r} p(s',r \mid s,a)\Big[r + \gamma v_{\pi'}(s')\Big].
\end{aligned}
$$

Value Iteration, for estimating $\pi \approx \pi_*$

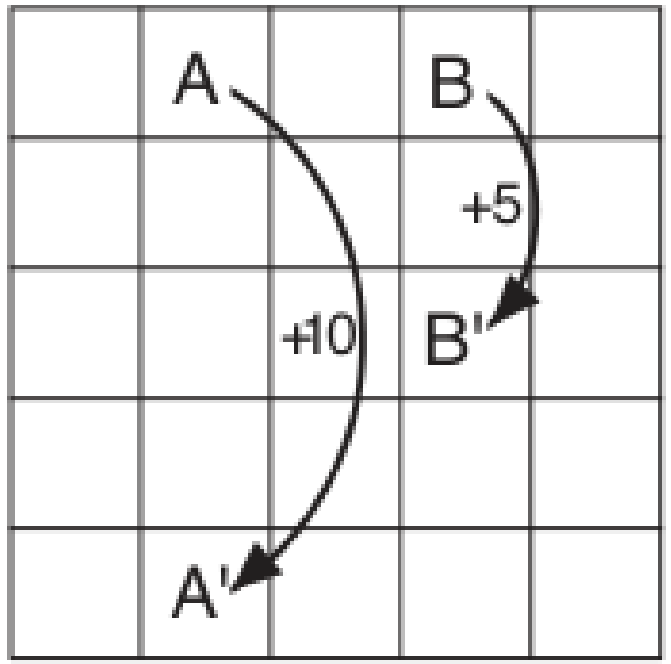
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma V(s')\big]$

Gridworld

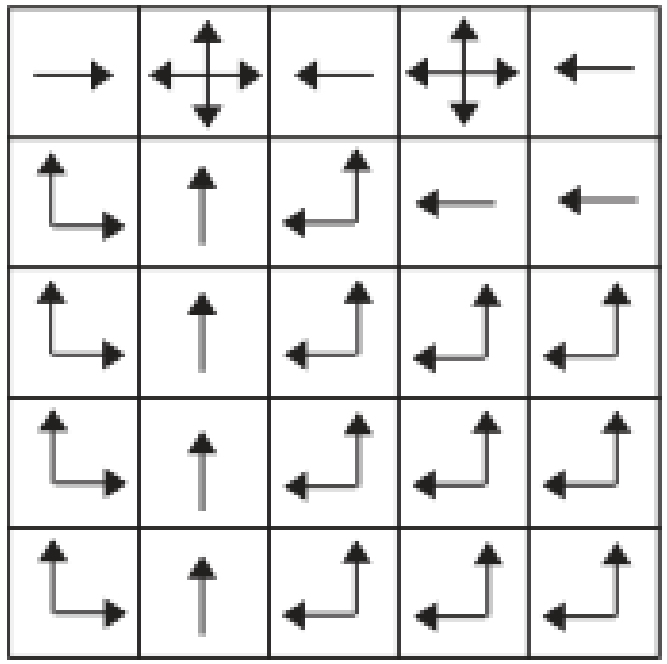
Задано поле рис (а), необхідно обчислити функцію цінності станів v_*(б) та визначити оптимальну політику π_* (в)



а)

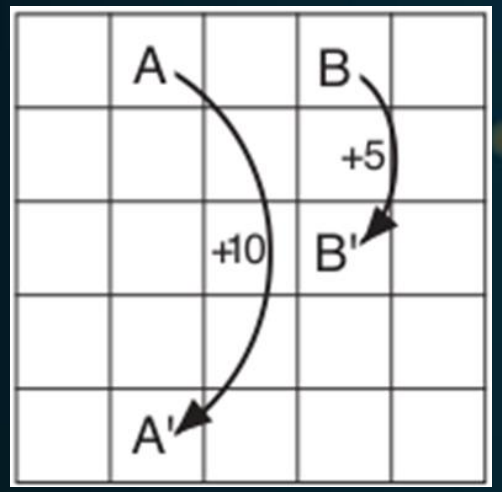
22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

б)



в)

Розв’язок



$$\gamma = 0.9$$

$$s = \overline{(0, 24)}$$

$$a = \{u, d, l, r\}$$

0	10	9	5	4.5
0	9	8.1	7.29	6.561
0	8.1	7.29	6.561	5.9049
0	7.29	6.561	5.9049	5.31441
0	6.561	5.9049	5.31441	4.782969

$v_*(0)=\max(0,10,-1,0)=10$

$v_*(1)=\max(0,10,-1,0)=10$

$v_*(2)=\max(0,9,-1,0)=9$

$v_*(3)=\max(0,5,-1,0)=5$

$v_*(4)=\max(0,4.5,-1,0)=4.5$

$v_*(5)=\max(0,10,-1,0)=10$

$v_*(6)=\max(0,9,-1,0)=9$

$v_*(7)=\max(0,5,-1,0)=5$

$v_*(8)=\max(0,4.5,-1,0)=4.5$

$v_*(9)=\max(0,10,-1,0)=10$

$v_*(10)=\max(0,9,-1,0)=9$

$v_*(11)=\max(0,5,-1,0)=5$

$v_*(12)=\max(0,4.5,-1,0)=4.5$

$v_*(13)=\max(0,10,-1,0)=10$

$v_*(14)=\max(0,9,-1,0)=9$

$v_*(15)=\max(0,5,-1,0)=5$

$v_*(16)=\max(0,4.5,-1,0)=4.5$

$v_*(17)=\max(0,10,-1,0)=10$

$v_*(18)=\max(0,9,-1,0)=9$

$v_*(19)=\max(0,5,-1,0)=5$

$v_*(20)=\max(0,4.5,-1,0)=4.5$

$v_*(21)=\max(0,10,-1,0)=10$

$v_*(22)=\max(0,9,-1,0)=9$

$v_*(23)=\max(0,5,-1,0)=5$

$v_*(24)=\max(0,4.5,-1,0)=4.5$

$v_*(0)=\max(10,10,-1,0)=10$

$v_*(1)=\max(10,10,-1,0)=10$

$v_*(2)=\max(9,9,-1,0)=9$

$v_*(3)=\max(5,5,-1,0)=5$

$v_*(4)=\max(4.5,4.5,-1,0)=4.5$

$v_*(5)=\max(10,10,-1,0)=10$

$v_*(6)=\max(9,9,-1,0)=9$

$v_*(7)=\max(5,5,-1,0)=5$

$v_*(8)=\max(4.5,4.5,-1,0)=4.5$

$v_*(9)=\max(10,10,-1,0)=10$

$v_*(10)=\max(9,9,-1,0)=9$

$v_*(11)=\max(5,5,-1,0)=5$

$v_*(12)=\max(4.5,4.5,-1,0)=4.5$

$v_*(13)=\max(10,10,-1,0)=10$

$v_*(14)=\max(9,9,-1,0)=9$

$v_*(15)=\max(5,5,-1,0)=5$

$v_*(16)=\max(4.5,4.5,-1,0)=4.5$

$v_*(17)=\max(10,10,-1,0)=10$

$v_*(18)=\max(9,9,-1,0)=9$

$v_*(19)=\max(5,5,-1,0)=5$

$v_*(20)=\max(4.5,4.5,-1,0)=4.5$

$v_*(21)=\max(10,10,-1,0)=10$

$v_*(22)=\max(9,9,-1,0)=9$

$v_*(23)=\max(5,5,-1,0)=5$

$v_*(24)=\max(4.5,4.5,-1,0)=4.5$

Агент може переміщатись по клітинкам поля вверх, вниз, вліво, вправо.

При переміщенні всередині поля винагорода =**0** для всіх клітинок (станів), крім: A (при виборі будь-якої дії винагорода **+10**, перехід у A’) та B (при виборі будь-якої дії винагорода **+5**, перехід у B’).

При виборі дії, виконання якої виведе агента за межі поля: винагорода **-1**, агент залишається на попередній позиції.

Завдання – максимізація винагороди.

```
itter: 1  val:
 [[ 0.        10.         9.         5.         4.5      ]
 [ 0.         9.         8.1        7.29       6.561    ]
 [ 0.         8.1        7.29       6.561      5.9049   ]
 [ 0.         7.29       6.561      5.9049     5.31441  ]
 [ 0.         6.561      5.9049     5.31441    4.782969]]
delta= 10.0
```

tre=0.0000000000001

```
itter: 1  val:
 [[ 0.        10.         9.         5.         4.5      ]
 [ 0.        9.         8.1        7.29       6.561    ]
 [ 0.        8.1        7.29       6.561      5.9049   ]
 [ 0.        7.29       6.561      5.9049     5.31441  ]
 [ 0.        6.561      5.9049     5.31441    4.782969]]
delta= 10.0
```

```
itter: 2  val:
 [[ 9.         15.9049     14.31441    10.9049      9.81441    ]
 [ 8.1        14.31441    12.882969   11.5946721  10.43520489]
 [ 7.29       12.882969   11.5946721  10.43520489  9.3916844  ]
 [ 6.561      11.5946721  10.43520489  9.3916844   8.45251596]
 [ 5.9049     10.43520489  9.3916844   8.45251596  7.60726436]]
delta= 9.0
```

```
itter: 3  val:
 [[14.31441    19.3916844  17.45251596 14.3916844  12.95251596]
 [12.882969   17.45251596 15.70726436 14.13653793 12.72288414]
 [11.5946721  15.70726436 14.13653793 12.72288414 11.45059572]
 [10.43520489 14.13653793 12.72288414 11.45059572 10.30553615]
 [ 9.3916844  12.72288414 11.45059572 10.30553615  9.27498253]]
delta= 5.314410000000002
```

```
itter: 4  val:
 [[17.45251596 21.45059572 19.30553615 16.45059572 14.80553615]
 [15.70726436 19.30553615 17.37498253 15.63748428 14.07373585]
 [14.13653793 17.37498253 15.63748428 14.07373585 12.66636227]
 [12.72288414 15.63748428 14.07373585 12.66636227 11.39972604]
 [11.45059572 14.07373585 12.66636227 11.39972604 10.25975344]]
delta= 3.138105960899999
```

```
itter: 5  val:
 [[19.30553615 22.66636227 20.39972604 17.66636227 15.89972604]
 [17.37498253 20.39972604 18.35975344 16.52377809 14.87140028]
 [15.63748428 18.35975344 16.52377809 14.87140028 13.38426026]
 [14.07373585 16.52377809 14.87140028 13.38426026 12.04583423]
 [12.66636227 14.87140028 13.38426026 12.04583423 10.84125081]]
delta= 1.8530201888518434
```

```
itter: 6  val:
 [[20.39972604 23.38426026 21.04583423 18.38426026 16.54583423]
 [18.35975344 21.04583423 18.94125081 17.04712573 15.34241315]
 [16.52377809 18.94125081 17.04712573 15.34241315 13.80817184]
 [14.87140028 17.04712573 15.34241315 13.80817184 12.42735465]
 [13.38426026 15.34241315 13.80817184 12.42735465 11.18461919]]
delta= 1.094189891315125
```

```
itter: 7  val:
 [[21.04583423 23.80817184 21.42735465 18.80817184 16.92735465]
 [18.94125081 21.42735465 19.28461919 17.35615727 15.62054154]
 [17.04712573 19.28461919 17.35615727 15.62054154 14.05848739]
 [15.34241315 17.35615727 15.62054154 14.05848739 12.65263865]
 [13.80817184 15.62054154 14.05848739 12.65263865 11.38737478]]
delta= 0.6461081889226641
```

```
itter: 8  val:
 [[21.42735465 24.05848739 21.65263865 19.05848739 17.15263865]
 [19.28461919 21.65263865 19.48737478 17.53863731 15.78477358]
 [17.35615727 19.48737478 17.53863731 15.78477358 14.20629622]
 [15.62054154 17.53863731 15.78477358 14.20629622 12.7856666 ]
 [14.05848739 15.78477358 14.20629622 12.7856666  11.50709994]]
delta= 0.38152042447694257
```

tre=0.0000000000001

```
itter: 9  val:
 [[21.65263865 24.20629622 21.7856666  19.20629622 17.2856666 ]
 [19.48737478 21.7856666  19.60709994 17.64638994 15.88175095]
 [17.53863731 19.60709994 17.64638994 15.88175095 14.29357585]
 [15.78477358 17.64638994 15.88175095 14.29357585 12.86421827]
 [14.20629622 15.88175095 14.29357585 12.86421827 11.57779644]]
delta= 0.22528399544939504
```

```
itter: 10  val:
 [[21.7856666  24.29357585 21.86421827 19.29357585 17.36421827]
 [19.60709994 21.86421827 19.67779644 17.7100168  15.93901512]
 [17.64638994 19.67779644 17.7100168  15.93901512 14.34511361]
 [15.88175095 17.7100168  15.93901512 14.34511361 12.91060225]
 [14.29357585 15.93901512 14.34511361 12.91060225 11.61954202]]
delta= 0.1330279464729145
```

```
itter: 11  val:
 [[21.86421827 24.34511361 21.91060225 19.34511361 17.41060225]
 [19.67779644 21.91060225 19.71954202 17.74758782 15.97282904]
 [17.7100168  19.71954202 17.74758782 15.97282904 14.37554613]
 [15.93901512 17.74758782 15.97282904 14.37554613 12.93799152]
 [14.34511361 15.97282904 14.37554613 12.93799152 11.64419237]]
delta= 0.07855167211278768
```

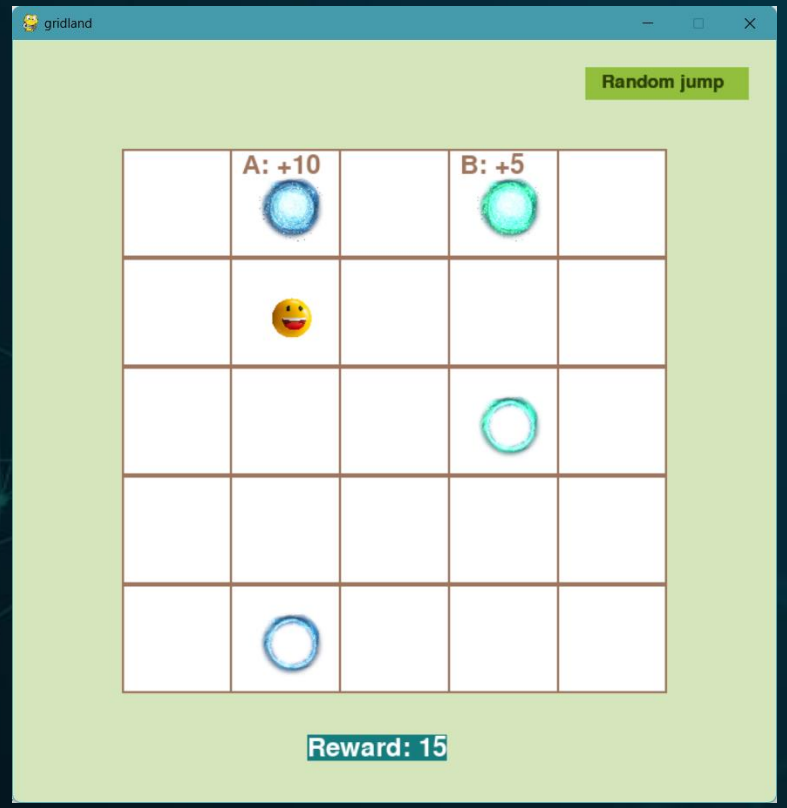
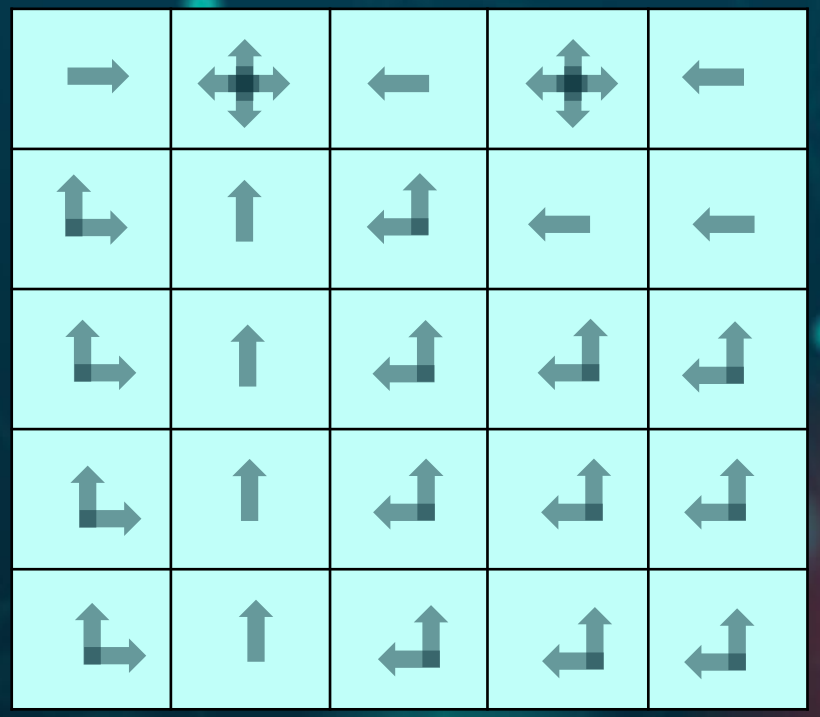
```
itter: 12  val:
 [[21.91060225 24.37554613 21.93799152 19.37554613 17.43799152]
 [19.71954202 21.93799152 19.74419237 17.76977313 15.99279582]
 [17.74758782 19.74419237 17.76977313 15.99279582 14.39351624]
 [15.97282904 17.76977313 15.99279582 14.39351624 12.95416461]
 [14.37554613 15.99279582 14.39351624 12.95416461 11.65874815]]
delta= 0.04638397686587936
```

```
itter: 13  val:
 [[21.93799152 24.39351624 21.95416461 19.39351624 17.45416461]
 [19.74419237 21.95416461 19.75874815 17.78287334 16.004586  ]
 [17.76977313 19.75874815 17.78287334 16.004586   14.4041274 ]
 [15.99279582 17.78287334 16.004586   14.4041274  12.96371466]
 [14.39351624 16.004586   14.4041274  12.96371466 11.6673432 ]]
delta= 0.027389274499533656
```

```
itter: 14  val:
 [[21.95416461 24.4041274  21.96371466 19.4041274  17.46371466]
 [19.75874815 21.96371466 19.7673432  17.79060888 16.01154799]
 [17.78287334 19.7673432  17.79060888 16.01154799 14.41039319]
 [16.004586   17.79060888 16.01154799 14.41039319 12.96935387]
 [14.4041274  16.01154799 14.41039319 12.96935387 11.67241848]]
delta= 0.01617309269922984
```

```
itter: 19  val:
 [[21.9758111  24.41832966 21.9764967  19.41832966 17.4764967 ]
 [19.77822999 21.9764967  19.77884703 17.80096232 16.02086609]
 [17.80040699 19.77884703 17.80096232 16.02086609 14.41877948]
 [16.02036629 17.80096232 16.02086609 14.41877948 12.97690153]
 [14.41832966 16.02086609 14.41877948 12.97690153 11.67921138]]
delta= 0.0011610630703522418
```

```
itter: 64  val:
 [[21.97748529 24.4194281  21.97748529 19.4194281  17.47748529]
 [19.77973676 21.97748529 19.77973676 17.80176308 16.02158677]
 [17.80176308 19.77973676 17.80176308 16.02158677 14.4194281 ]
 [16.02158677 17.80176308 16.02158677 14.4194281  12.97748529]
 [14.4194281  16.02158677 14.4194281  12.97748529 11.67973676]]
delta= 5.684341886080802e-14
```

Визначення π_*

```
itter: 64  val:
 [[21.97748529 24.4194281  21.97748529 19.4194281  17.47748529]
 [19.77973676 21.97748529 19.77973676 17.80176308 16.02158677]
 [17.80176308 19.77973676 17.80176308 16.02158677 14.4194281 ]
 [16.02158677 17.80176308 16.02158677 14.4194281  12.97748529]
 [14.4194281  16.02158677 14.4194281  12.97748529 11.67973676]]
delta= 5.684341886080802e-14
```